

# Depth, Residuals, Normalization, and CNN Architecture Math

Mathematics of Deep Learning — Chapter 6

---

Yin Yang

Foundation Books Project

# Roadmap

Architectures as Compositions

Tensor-Shape Accounting

Parameter and Compute Counts

Depth and Receptive Fields

Downsampling and Channel Growth

Normalization Layers

Residuals and Gradient Flow

Bottlenecks and Efficient Blocks

Architecture Motifs

Summary

# Architectures as Compositions

---

# From Layers to Blocks

A CNN architecture is a composition of *block maps*:

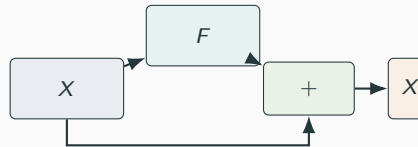
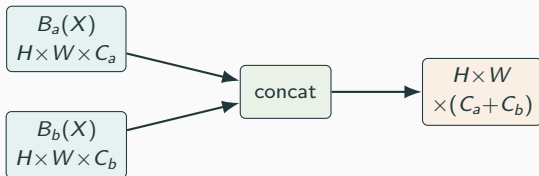
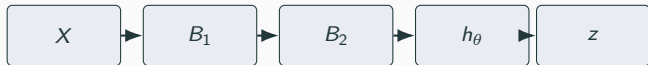
$$f_{\theta} = h_{\theta} \circ B_L \circ B_{L-1} \circ \cdots \circ B_1, \quad A_{\ell} = B_{\ell}(A_{\ell-1}).$$

- Each  $B_{\ell}$  may contain several internal layers (conv, normalization, activation).
- $h_{\theta}$ : classifier head — e.g. global average pool then affine  $z = Wv + b$ .
- Total parameters:  $\theta = (\theta_{B_1}, \dots, \theta_{B_L}, \theta_h)$ .

A *stage* is a run of blocks that share spatial size and channel count; a transition block changes them.

Treat the block — not the single layer — as the architecture unit.

# Architecture Graph: Chain, Branch, Shortcut



residual addition

Concatenation: match  $H, W$ ; sum channels. Addition: match every axis.

Each graph operation enforces a shape rule before any training begins.

# Tensor-Shape Accounting

---

## Convolution Output-Size Formula

For one spatial axis with input length  $N$ , kernel  $K$ , padding  $P$ , stride  $S$ , dilation  $D$ :

$$N_{\text{out}} = \left\lfloor \frac{N + 2P - (D(K - 1) + 1)}{S} + 1 \right\rfloor.$$

Apply separately to height and width. Channel count  $C_{\text{out}}$  is a designer choice.

*Shape ledger example.* A block maps  $32 \times 32 \times 64 \rightarrow 16 \times 16 \times 128$ :

$$32 \cdot 32 \cdot 64 = 65536 \quad \longrightarrow \quad 16 \cdot 16 \cdot 128 = 32768.$$

Spatial size halves; channels double; activation count halves.

Treat spatial axes and channel axes separately — most CNN bugs mix them.

# Graph-Operation Shape Rules

| Operation          | Shape condition                           | Applied check                                                           |
|--------------------|-------------------------------------------|-------------------------------------------------------------------------|
| Two-path addition  | Both paths share $H, W, C$                | Insert a learned shape map when a stage changes resolution or channels. |
| Channel concat     | Branches match $H$ and $W$                | Output channel count is the sum of branch channels.                     |
| Downsampling block | Next stage must expect new $H, W, C$      | Track activation count after downsampling.                              |
| Classifier head    | Final feature map must reduce to a vector | Global average pool removes spatial axes before the affine classifier.  |

Skip connections are unforgiving:  $16 \times 16 \times 64$  cannot be added to  $16 \times 16 \times 128$  without a projection.

A CNN architecture is not well-defined until every composition, concatenation, and addition has compatible shapes.



## Parameter and Compute Counts

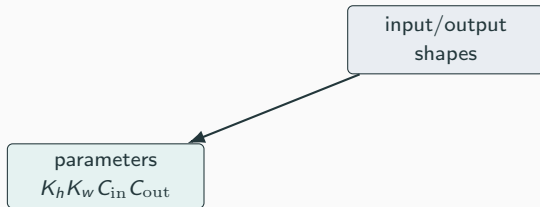
---

# Three Ledgers for One Convolution

input/output  
shapes

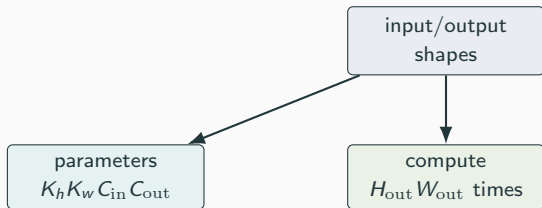
(1) Shapes pin down everything else.

# Three Ledgers for One Convolution



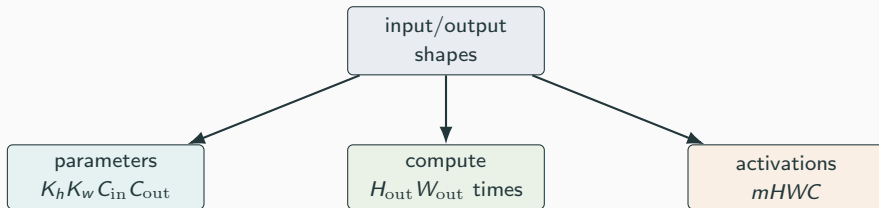
- (1) Shapes pin down everything else.
- (2) Parameters:  $K_h K_w C_{in} C_{out} + C_{out}$  (bias).

# Three Ledgers for One Convolution



- (1) Shapes pin down everything else.
- (2) Parameters:  $K_h K_w C_{in} C_{out} + C_{out}$  (bias).
- (3) Compute: also multiply by output spatial positions  $H_{out} W_{out}$ .

# Three Ledgers for One Convolution



- (1) Shapes pin down everything else.
- (2) Parameters:  $K_h K_w C_{in} C_{out} + C_{out}$  (bias).
- (3) Compute: also multiply by output spatial positions  $H_{out} W_{out}$ .
- (4) Activation memory depends on batch and feature map.

Right shape is necessary; cost decides whether the model is feasible.

## Early Layers Are Expensive

Compare two  $3 \times 3$  convolutions, both  $64 \rightarrow 64$  channels:

$$\text{at } 32 \times 32 : \quad 32 \cdot 32 \cdot 3 \cdot 3 \cdot 64 \cdot 64,$$

$$\text{at } 16 \times 16 : \quad 16 \cdot 16 \cdot 3 \cdot 3 \cdot 64 \cdot 64 = \frac{1}{4} \text{ of the cost.}$$

Same parameters; very different compute.

- *Width* = channel count in a stage. Raising width raises capacity *and* parameters, activations, and compute.
- Modern CNNs stack small kernels:  $5 \times 5$  has 25 spatial weights per channel pair,  $3 \times 3$  has 9.

Downsampling cuts compute even when parameter count does not change.

## Depth and Receptive Fields

---

# Receptive-Field Recurrence

Initialize  $R_0 = 1$ ,  $J_0 = 1$ . For layer  $\ell$  with kernel  $K_\ell$  and stride  $S_\ell$ :

$$R_\ell = R_{\ell-1} + (K_\ell - 1) J_{\ell-1}, \quad J_\ell = J_{\ell-1} S_\ell.$$

$R_\ell$  is the input region one activation can see;  $J_\ell$  is the input-space spacing between neighboring activations.

*Stride changes the jump.* A 3-wide stride-1 layer then a 3-wide stride-2 layer:

$$R_1 = 3, J_1 = 1; \quad R_2 = 3 + (3-1)(1) = 5, J_2 = 2.$$

With dilation  $D_\ell$ : replace  $(K_\ell - 1)$  by  $(K_\ell - 1)D_\ell$ .

Depth widens context: later activations depend on larger input regions.



# Theoretical vs. Effective Receptive Field

layer  $L$



theoretical RF: width  $2L + 1$

(1) Theoretical RF: which positions *can* affect the output.

# Theoretical vs. Effective Receptive Field

layer  $L$

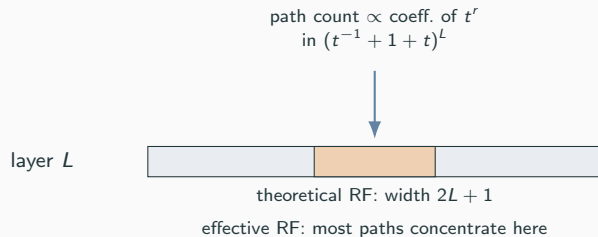


theoretical RF: width  $2L + 1$

effective RF: most paths concentrate here

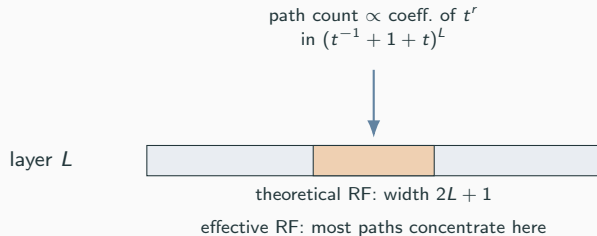
- (1) Theoretical RF: which positions *can* affect the output.
- (2) Effective RF: which positions *do* influence it.

# Theoretical vs. Effective Receptive Field



- (1) Theoretical RF: which positions *can* affect the output.
- (2) Effective RF: which positions *do* influence it.
- (3) Paths to far offsets are rare; nearby offsets are reached by many paths.

# Theoretical vs. Effective Receptive Field



Central-limit intuition: random walks of length  $L$  concentrate  $\sim \sqrt{L}$  around 0.

- (1) Theoretical RF: which positions *can* affect the output.
- (2) Effective RF: which positions *do* influence it.
- (3) Paths to far offsets are rare; nearby offsets are reached by many paths.
- (4) Depth alone does not make distant pixels equally important.

Theoretical RF  $\neq$  effective RF; the bell-shaped weighting matters in practice.

## Two Small Kernels vs. One Large One

Two stride-1,  $3 \times 3$  layers:

$$R_2 = 1 + (3-1) + (3-1) = 5.$$

Compared to a single  $5 \times 5$  layer (with  $C_{\text{in}} = C_{\text{out}} = C$ ):

|                  | Kernel weights | Receptive field |
|------------------|----------------|-----------------|
| one $5 \times 5$ | $25C^2$        | $5 \times 5$    |
| two $3 \times 3$ | $18C^2$        | $5 \times 5$    |

Two-layer version also inserts a nonlinearity between the linear pieces.

Depth can match receptive field at lower parameter cost — and with extra nonlinear capacity.

## Downsampling and Channel Growth

---

# The Classic Stage Transition

Many CNNs use shape maps of the form

$$H \times W \times C \mapsto \frac{H}{2} \times \frac{W}{2} \times 2C.$$

Activation count before:  $HWC$ . After:

$$\frac{H}{2} \frac{W}{2} (2C) = \frac{1}{2} HWC.$$

- Total activations *cut in half* per image.
- Channels double: room for richer per-position features.
- Compute on the next  $3 \times 3$  convolution drops by roughly  $4 \times$  from the spatial shrink, but doubles back from each channel direction.

Trade spatial detail for semantic abstraction — and save on activation memory.

## Linear Downsampling Has a Nullspace

View a transition  $H \times W \times C \rightarrow \frac{H}{2} \times \frac{W}{2} \times 2C$  as a linear map

$$D : \mathbb{R}^{HWC} \rightarrow \mathbb{R}^{HWC/2}.$$

Rank-nullity:  $\dim \ker(D) = HWC - \text{rank}(D)$ . Since  $\text{rank}(D) \leq HWC/2$ ,

$$\dim \ker(D) \geq \frac{1}{2}HWC.$$

So there exist nonzero  $u$  with  $D(u) = 0$ , and for every activation  $a$ :  $D(a + u) = D(a)$ .

Some changes to detail are *invisible* after the transition — the map is not one-to-one.

Downsampling discards degrees of freedom; choose what to keep.



# Normalization Layers

---

# Batch Normalization (Per Channel)

For channel  $c$  with statistic set  $S_c$ :

$$\mu_c = \frac{1}{|S_c|} \sum_{\alpha \in S_c} x_{\alpha,c}, \quad \sigma_c^2 = \frac{1}{|S_c|} \sum_{\alpha \in S_c} (x_{\alpha,c} - \mu_c)^2.$$

Transform each activation in that channel:

$$\hat{x}_{\alpha,c} = \frac{x_{\alpha,c} - \mu_c}{\sqrt{\sigma_c^2 + \epsilon}}, \quad y_{\alpha,c} = \gamma_c \hat{x}_{\alpha,c} + \beta_c.$$

- For conv maps,  $S_c$  runs over batch *and* spatial axes.
- Learned  $\gamma_c, \beta_c$ : restore useful scale and offset.
- Inference uses *stored* running estimates, not the current batch.

Normalize per channel; then learn the scale and shift back if useful.

# Normalizing Four Numbers

Activations 2, 4, 6, 8. Mean  $\mu = 5$ . Population variance:

$$\sigma^2 = \frac{1}{4}((2-5)^2 + (4-5)^2 + (6-5)^2 + (8-5)^2) = 5.$$

With  $\epsilon = 0$ ,  $\gamma = 1$ ,  $\beta = 0$ :

$$\hat{x} \in \left\{ \frac{-3}{\sqrt{5}}, \frac{-1}{\sqrt{5}}, \frac{1}{\sqrt{5}}, \frac{3}{\sqrt{5}} \right\}.$$

Check: sample mean 0, sample variance  $\sigma^2/(\sigma^2 + \epsilon) = 1$ .

- Centering is the average of  $x_i - \mu$ ; scaling divides by a single constant per channel.
- The convolution bias is often dropped:  $\beta_c$  already absorbs it.

Per-channel statistics give every channel a stable numerical problem for the next layer.

## Training Mode Couples the Batch

Backward through batch norm (one channel,  $s = \sqrt{\sigma^2 + \epsilon}$ ):

$$\frac{\partial L}{\partial \beta} = \sum_i g_i, \quad \frac{\partial L}{\partial \gamma} = \sum_i g_i \hat{x}_i.$$

Input gradient is *not* elementwise:

$$\frac{\partial L}{\partial x_i} = \frac{\gamma}{s} \left( g_i - \bar{g} - \hat{x}_i \overline{g\hat{x}} \right),$$

with  $\bar{g} = \frac{1}{N} \sum_j g_j$  and  $\overline{g\hat{x}} = \frac{1}{N} \sum_j g_j \hat{x}_j$ .

Inference mode (fixed  $s_{\text{run}}$ ) reduces to the affine  $\partial L / \partial x_i = (\gamma / s_{\text{run}}) g_i$ .

Training-mode batch norm is not an independent elementwise map — the batch couples gradients.

# Residuals and Gradient Flow

---

# The Residual Block

$$y = x + F_{\theta}(x).$$

- $F_{\theta}(x) = 0$  recovers exactly the identity.
- Learned path adds an *increment*, instead of replacing  $x$ .
- Requires  $\text{shape}(F_{\theta}(x)) = \text{shape}(x)$ .

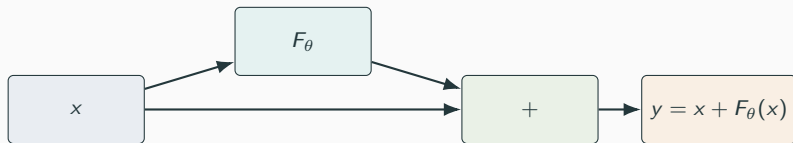
*Scalar example.*  $F_a(x) = ax^3$ , so  $y = x + ax^3$  and

$$\frac{dy}{dx} = 1 + 3ax^2.$$

At  $x = 0.5$ ,  $a = 0.2$ : derivative = 1.15. Plain path  $y = ax^3$  would give 0.15 at the same point — the shortcut supplies the +1.

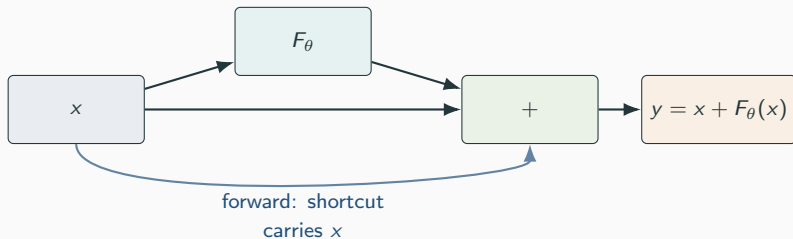
The shortcut puts the identity inside the block's function class.

# Gradient Through One Residual Block



(1) Shortcut and learned path both feed the sum.

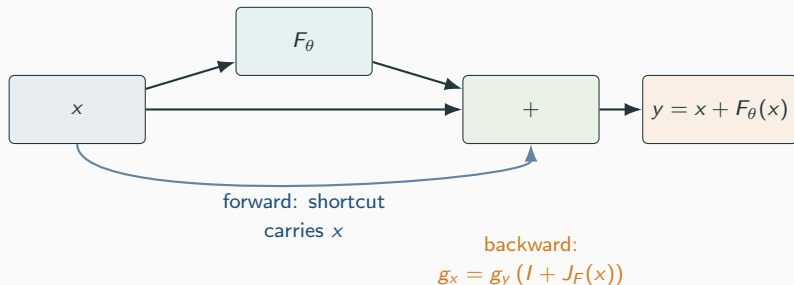
# Gradient Through One Residual Block



- (1) Shortcut and learned path both feed the sum.
- (2) Forward: shortcut just copies  $x$ .

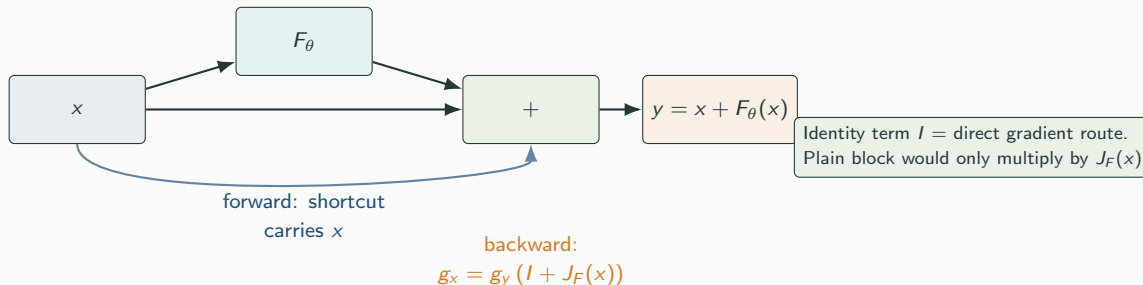


# Gradient Through One Residual Block



- (1) Shortcut and learned path both feed the sum.
- (2) Forward: shortcut just copies  $x$ .
- (3) Backward differential:  $dy = (I + J_F(x)) dx$ .

# Gradient Through One Residual Block



- (1) Shortcut and learned path both feed the sum.
- (2) Forward: shortcut just copies  $x$ .
- (3) Backward differential:  $dy = (I + J_F(x)) dx$ .
- (4) The identity term is the Jacobian signature of the shortcut.

Residual block multiplies the upstream gradient by  $I + J_F$ , not just  $J_F$ .

# Stacking Residual Blocks

$x_{k+1} = x_k + F_k(x_k)$ . Then

$$g_k = g_{k+1}(I + J_{F_k}(x_k)) = g_{k+1} + g_{k+1} J_{F_k}(x_k).$$

First term: gradient carried *directly* through the shortcut. Second term: gradient through the learned residual path.

Two stacked blocks:

$$g_k = g_{k+2}(I + J_{F_{k+1}})(I + J_{F_k}) = g_{k+2} + g_{k+2}J_{F_{k+1}} + g_{k+2}J_{F_k} + g_{k+2}J_{F_{k+1}}J_{F_k}.$$

Identity contribution survives every multiplication; plain chains have no such direct route.

The straight-through identity path is why very deep residual networks remain optimizable.

# Projection Shortcut for Shape Mismatch

When  $\text{shape}(x) \neq \text{shape}(F_\theta(x))$ :

$$y = P_\phi(x) + F_\theta(x).$$

Common projection:  $1 \times 1$  convolution that adjusts channels and/or stride.

*Example.*  $x \in \mathbb{R}^{16 \times 16 \times 32}$ ,  $F_\theta(x) \in \mathbb{R}^{16 \times 16 \times 64}$ :

- Direct sum  $x + F_\theta(x)$  undefined.
- $1 \times 1$  convolution  $P_\phi : 32 \rightarrow 64$  maps  $x$  to  $16 \times 16 \times 64$ .
- Now  $P_\phi(x) + F_\theta(x)$  is well-defined.

Residual additions are unforgiving about shape; projections fix stage transitions.

## Bottlenecks and Efficient Blocks

---

# Pointwise Convolution Mixes Channels

A pointwise ( $1 \times 1$ ) convolution applies the same affine map at every spatial position:

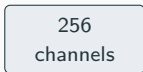
$$v = Wu + b, \quad W \in \mathbb{R}^{C_{\text{out}} \times C_{\text{in}}}, \quad u \in \mathbb{R}^{C_{\text{in}}}.$$

- Changes channel count.
- Does *not* directly mix neighboring pixels.
- Acts as a *channel mixer*.

Parameters per pointwise layer:  $C_{\text{in}} C_{\text{out}}$  (no  $K^2$  factor).

Pointwise convolutions are the cheap channel-reshape tool inside bottlenecks and depthwise-separable blocks.

# Bottleneck Block



**Direct**  $3 \times 3$  at  $256 \rightarrow 256$ :  $3 \cdot 3 \cdot 256 \cdot 256 = 589,824$  weights.

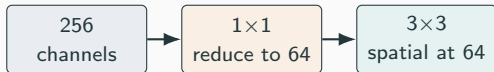
## Bottleneck Block



**Direct**  $3 \times 3$  at  $256 \rightarrow 256$ :  $3 \cdot 3 \cdot 256 \cdot 256 = 589,824$  weights.



## Bottleneck Block



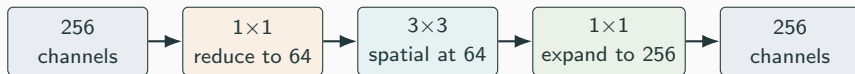
**Direct**  $3 \times 3$  at  $256 \rightarrow 256$ :  $3 \cdot 3 \cdot 256 \cdot 256 = 589,824$  weights.

## Bottleneck Block



**Direct**  $3 \times 3$  at  $256 \rightarrow 256$ :  $3 \cdot 3 \cdot 256 \cdot 256 = 589,824$  weights.

## Bottleneck Block



**Direct**  $3 \times 3$  at  $256 \rightarrow 256$ :  $3 \cdot 3 \cdot 256 \cdot 256 = 589,824$  weights.

**Bottleneck:**  $1 \cdot 1 \cdot 256 \cdot 64 + 3 \cdot 3 \cdot 64 \cdot 64 + 1 \cdot 1 \cdot 64 \cdot 256 = 69,632$ .

Bottleneck factorization changes the function class to win parameter cost — roughly  $8\times$  in this example.

# Depthwise Separable Convolution

Factor a  $K \times K$  kernel into spatial-only and channel-only parts:

$$W_{u,v,c,r} = D_{u,v,c} P_{c,r}.$$

- Depthwise:  $K^2 C_{\text{in}}$  weights (one  $K \times K$  filter per input channel).
- Pointwise:  $C_{\text{in}} C_{\text{out}}$  weights (channel mix).
- Total:  $K^2 C_{\text{in}} + C_{\text{in}} C_{\text{out}}$ .

Ratio to standard  $K^2 C_{\text{in}} C_{\text{out}}$ :

$$\frac{1}{C_{\text{out}}} + \frac{1}{K^2}.$$

$K = 3$ ,  $C_{\text{in}} = 128$ ,  $C_{\text{out}} = 256$ :  $294,912 \rightarrow 33,920$  weights.

Fewer parameters come from a factorization constraint, not from computing the same dense convolution more cleverly.

# Architecture Motifs

---

# Translation Equivariance Forces Convolution

Circular shift  $T_s$ :  $(T_s x)_i = x_{i-s \bmod n}$ . If a linear map  $A$  commutes with every shift,

$$AT_s = T_s A \quad \text{for all } s,$$

then  $A$  is circular convolution with some kernel  $a$ :

$$(Ax)_i = \sum_{j=0}^{n-1} a_{i-j \bmod n} x_j.$$

*Sketch.*  $e_j = T_j e_0$ , so  $Ae_j = T_j(Ae_0)$ . Every column of  $A$  is a shift of the first column — the matrix of convolution.

Two-dimensional image case is analogous; group-equivariant CNNs extend this to richer symmetries.

Translation equivariance is not a CNN convention — it *forces* the convolutional shared-weight form.

# Compound Scaling

Leading compute term of a standard convolution:

$$H W k^2 C_{\text{in}} C_{\text{out}}.$$

- Depth  $\times \delta \Rightarrow \text{compute} \times \delta$ .
- Width  $\times \alpha$  (both  $C_{\text{in}}$  and  $C_{\text{out}}$ )  $\Rightarrow \text{compute} \times \alpha^2$ .
- Resolution  $\times \rho$  (both  $H$  and  $W$ )  $\Rightarrow \text{compute} \times \rho^2$ .

Compound scaling couples all three knobs instead of moving one at a time:

$$\delta \cdot \alpha^2 \cdot \rho^2 \approx \text{target compute factor}.$$

Compute scales as a product; a balanced increase across depth, width, and resolution is one design strategy.

# Motif Vocabulary, Read Mathematically

| Motif                  | Mathematical content                                                                                                       |
|------------------------|----------------------------------------------------------------------------------------------------------------------------|
| Repeated small kernels | RF $R_L = 1 + \sum (K_\ell - 1)$ ; two $3 \times 3$ replace one $5 \times 5$ with fewer weights and an added nonlinearity. |
| Multi-branch concat    | Branches share $H, W$ ; output channel count is the sum.                                                                   |
| Residual addition      | Block Jacobian $I + J_F(X)$ ; identity path preserved in the gradient.                                                     |
| Depthwise separable    | Kernel factorization $W_{u,v,c,r} = D_{u,v,c} P_{c,r}$ .                                                                   |
| Compound scaling       | Couples depth ( $\delta$ ), width ( $\alpha^2$ ), and resolution ( $\rho^2$ ) factors of compute.                          |

Translate each architecture name into a shape map, graph operation, factorization, or gradient path.

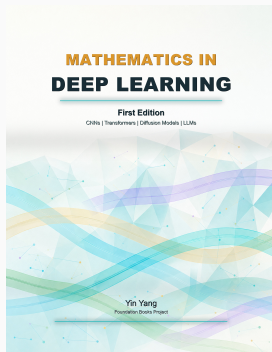


# Summary

---

- **Compositions:** a CNN is a chain of *blocks*; graph operations (concat, add) have shape rules.
- **Shape ledger:** apply  $N_{\text{out}} = \lfloor (N + 2P - (D(K - 1) + 1))/S + 1 \rfloor$  per axis.
- **Cost ledger:** parameters  $K^2 C_{\text{in}} C_{\text{out}}$ , compute  $\cdot H_{\text{out}} W_{\text{out}}$ , activations  $mHWC$ .
- **Receptive field:**  $R_\ell = R_{\ell-1} + (K_\ell - 1)J_{\ell-1}$ ,  $J_\ell = J_{\ell-1}S_\ell$ ; effective  $\neq$  theoretical.
- **Downsampling:**  $\frac{H}{2} \frac{W}{2} (2C) = \frac{HWC}{2}$ ; a linear downsampling map has a nullspace.
- **Normalization:** per-channel  $\gamma_c \hat{x} + \beta_c$ ; training-mode backward is batch-coupled.
- **Residuals:**  $y = x + F(x)$  gives Jacobian  $I + J_F$  — a direct gradient route through depth.
- **Efficient blocks:** pointwise mixes channels; bottlenecks and depthwise-separable factorizations are parameter-class constraints.

**Next: from CNN families to the data and training questions that decide which architecture wins.**



## Mathematics in Deep Learning

Chapter overview slides for the book by Yin Yang.

|           |                                                                                                                                                   |
|-----------|---------------------------------------------------------------------------------------------------------------------------------------------------|
| Book page | <a href="https://foundation-books.github.io/mathematics-in-deep-learning">foundation-books.github.io/mathematics-in-deep-learning</a>             |
| Code      | <a href="https://github.com/foundation-books/mathematics-in-deep-learning-code">github.com/foundation-books/mathematics-in-deep-learning-code</a> |
| Paperback | <a href="https://amazon.com/dp/BOH1ZZHDT2">amazon.com/dp/BOH1ZZHDT2</a>                                                                           |
| Hardcover | <a href="https://amazon.com/dp/BOH1ZL11MJ">amazon.com/dp/BOH1ZL11MJ</a>                                                                           |
| Kindle    | <a href="https://amazon.com/dp/B0GX32M8SP">amazon.com/dp/B0GX32M8SP</a>                                                                           |
| License   | CC BY-NC-ND 4.0                                                                                                                                   |

These free overview slides may be shared non-commercially with attribution and without modifications. Full explanations, exercises, and companion-code context are in the book and linked repository.